

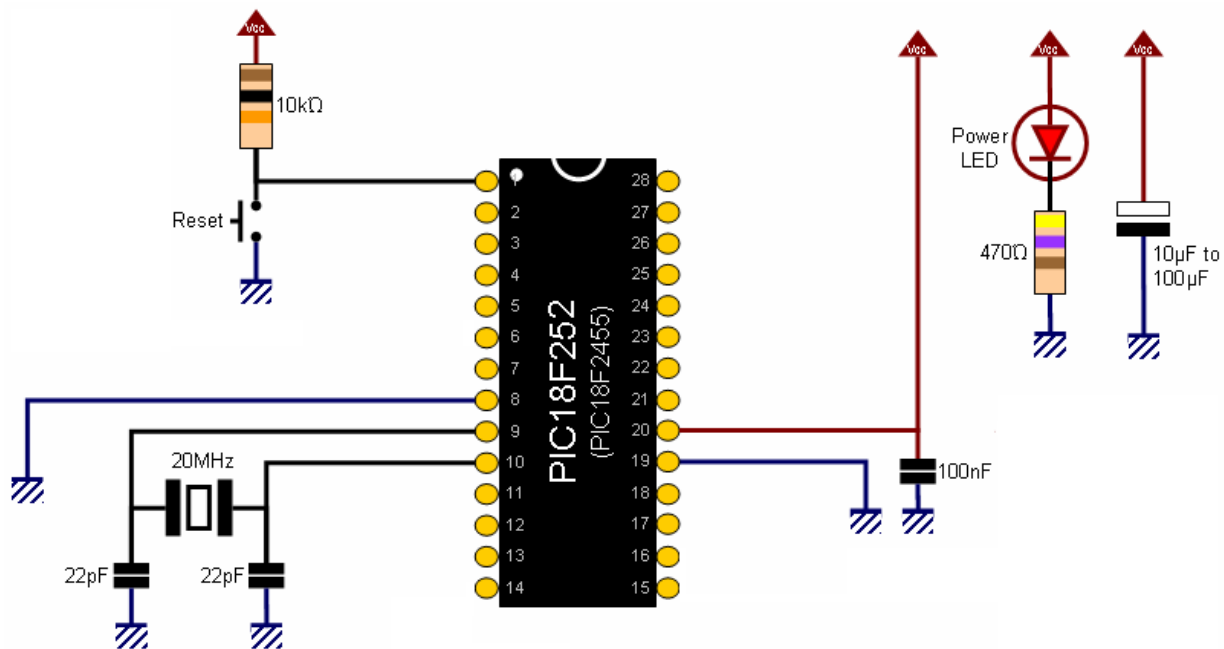
TALLER V - MICROCONTROLADORES

Escuela de Física - Facultad de Ciencias

Universidad Nacional de Colombia - Sede Medellín - 2017

PROFESOR: NERIO ANDRÉS MONTOYA - CARLOS TRUJILLO

namontoy@unal.edu.co - catrujila@unal.edu.co



Examen Final - Taller V

Imagen #1 - Montaje básico

Objetivos

- Evaluar la capacidad de cada estudiante para programar un microcontrolador.

-
- Utilizar el microcontrolador en un modo diferente al utilizado en el desarrollo del curso.
 - Verificar que se tienen los conocimientos mínimos para trabajar con las diferentes características de un microcontrolador.
 - Aprender un método para cambiar el comportamiento del microcontrolador sin tener la necesidad de reprogramar el micro.

Introducción

El siguiente examen es un poco “particular” en el sentido que es una actividad guiada, con la que se desea que el estudiante llegue más lejos de los conocimientos previos adquiridos en el curso y con la que puede profundizar en el manejo de los microcontroladores, mientras es evaluado.

Para desarrollar esta actividad, cada estudiante trabajará con su microcontrolador (PIC18F252), pero algunas otros elementos (como el cristal) van a ser diferentes y/o nuevos, para generar la necesidad en el estudiante de reevaluar todo el conocimiento adquirido durante el curso y someterlo a una realidad del día a día en la vida fuera de la U, donde la necesidad de utilizar otros dispositivos diferentes a los conocidos es lo más común.

Además del microcontrolador y el cristal, al estudiante se le entregará un programa básico, desde el cual debe partir para desarrollar toda la actividad evaluativa.

Todos los pasos descritos a continuación se hacen basados en el ambiente de desarrollo **MPLAB-X v3.40** (compilador XC8 v1.38), el sistema operativo **Linux OpenSUSE 13.2**, el programador **Pickit 3** y la interface de comunicación serial **minicom** (terminal).

Para desarrollar la actividad, cada estudiante requiere:

- PIC18F252
- Cristal de 10MHz
- 6 Potenciómetros Tipo B, de un valor entre 5K y 20K (los 6 pueden ser diferentes)

-
- Demultiplexor 74HC139 (también es posible utilizar el 74LS139)
 - Compuerta Inversora (puede utilizar el Schmitt Trigger de la caja)
 - Elementos variados como resistencias, leds, capacitores y demás items necesarios para hacer un adecuado montaje

El objetivo final del programa que se desea desarrollar es que al micro se le puedan enviar comandos desde una interface serial y que el micro responda adecuadamente. Este tipo de comandos son elementos más complejos que simples envíos de teclas como 'j' o 'm', pasando a ser cadenas de caracteres con las cuales se puede tener comandos que reciben parámetros, los cuales pueden modificar el funcionamiento del micro sin la necesidad de reprogramarlo.

Para el caso en cuestión y debido a la necesidad de cumplir con algunas restricciones necesarias para el funcionamiento adecuado del equipo, todos los comandos que se van a ejecutar serán del tipo: "comando número número", por ejemplo:

```
setADC 0 10<enter>
```

Nota: El símbolo <enter> significa que se presiona la tecla Enter.

Observe que en el ejemplo, los números van separados por un (1) símbolo espacio. Esta convención debe ser respetada siempre. Dado el caso que se tenga un comando que no requiera numeros, entonces se podrá introducir dos valores cualquiera, ya que el programa deberá ser capaz de simplemente no hacer nada con ellos. Por defecto, estos valores (si no se necesitan) pueden ser el valor cero (0). Imagine por ejemplo que tiene un comando llamado setPin, el cual requiere solo un parámetro con valor cero (0) o uno (1), este comando se puede escribir como:

```
setPin 1 35<enter>
```

Y se debe obtener como resultado que el pin deseado se enciende y el segundo valor simplemente es omitido.

También puede pensarse que el primer parámetro sea el número del pin y el segundo sea el estado (1 -> encendido, 2 -> apagado), de esta forma el comando

setPin 3 1<enter>

Indicaría que el pin "3" debe ser puesto en 1 lógico, o sea encendido.

Imagine un comando que sea "setLedState 250 0<enter>". Podría significar que le asignamos un periodo de 250ms a un timer que nos controla el led de estado.

NOTA: Para el caso de minicom, lo debe configurar para que cada vez que se presione la tecla "enter", el minicom envíe los caracteres LF+CR, para esto debe entrar a minicom y cuando este ya recibiendo datos del micro abrir todas las opciones (teclas CTRL+A y luego la opción Z). Se debe desplegar un menú con muchas opciones (ver imagen 2), entre las cuales está U, que es para activar el envío del CR. La imagen muestra las opciones del minicom, en la esquina inferior izquierda se ve el comando U.

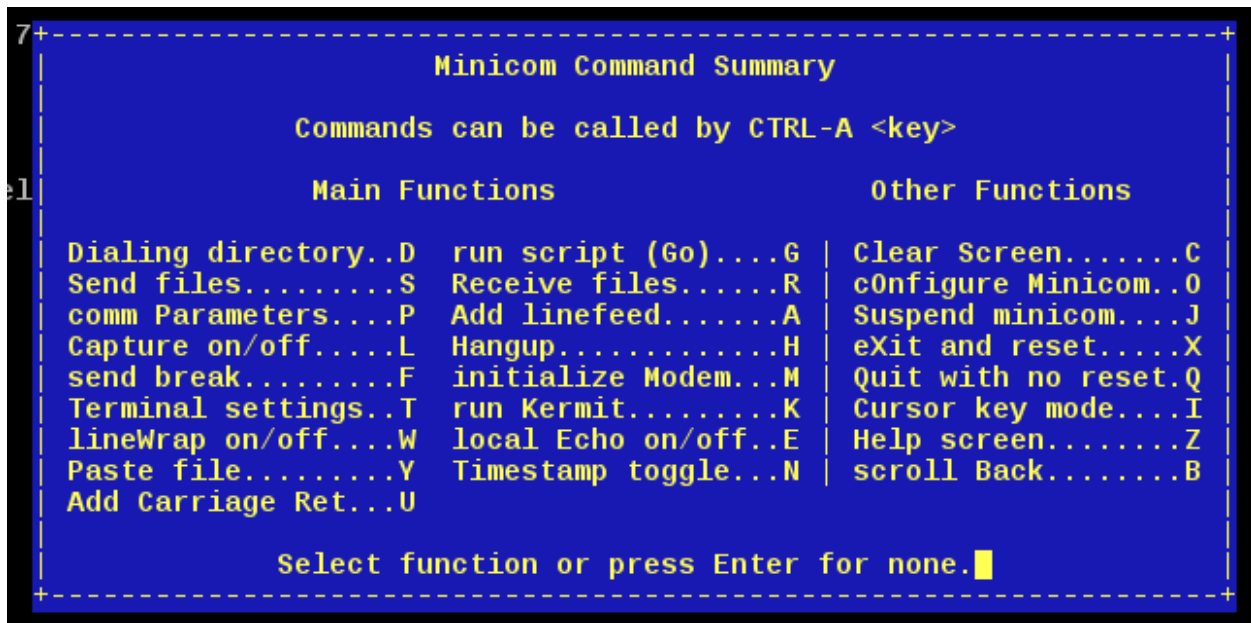


Imagen #2 - Menú minicom

Procedimiento

Parte A - 50%

A continuación se describen los pasos a seguir para desarrollar los puntos del examen. Si usted sigue con juicio estos puntos, con seguridad podrá llevar a cabo y terminar la tarea felizmente.

A no ser que se indique otra cosa, todos los pasos descritos relacionados con programación, deben ser desarrollados en el código de la clase main:

1. Instale su microcontrolador en la board, conectando adecuadamente los pines de alimentación (Vdd y GND), el cristal y sus capacitores, el MCLR, y los pines asociados a la programación. También tenga en cuenta los pines para la comunicación serial con su respectivo cable USB-Serial.
2. Abra el proyecto que se le ha entregado y cambie el nombre del proyecto, en la parte donde dice "NombreEstudiante" ponga su nombre.
3. Genere los bits de configuración adecuados teniendo en cuenta el cristal (10 MHz, tipo HS), además de tener en cuenta todas las otras configuraciones, como desactivar el Watch Dog Timer, configurar el Power On Timer, y el Brown Out.
4. Active el módulo PLL para que el equipo quede funcionando a 40 MHz (bits de configuración)
5. Verifique que el programa compile. No debería tener ningún warning "grave" u otros inconvenientes.
6. Organice el código correspondiente en el puerto serial para que el equipo Transmita/Reciba datos a 9600bps en configuración 8N1 (recuerde que su velocidad ahora es de 40 MHz y no de 20 MHz, por lo cual debe recalcular todo).
7. Organice el código correspondiente al módulo del led de estado para que este tenga un periodo de 500ms (o sea 250ms encendido y 250ms apagado).

-
8. Verifique que las librerías <stdbool.h>, <string.h>, <stdio.h> están incluidas en el main.
 9. Cree las variables que se presentan en la imagen de la siguiente página. Por favor copie los nombres, tipos y tamaños iguales.
 10. Observe que su led de estado debe estar parpadeando y el micro ya debe estar en la capacidad de enviar el mensaje de saludo. Dentro del while infinito agregue dos nuevos bloques **if()** independientes, uno para cuando la variable receptionChar es diferente del carácter NULL y el otro para cuando la variable receptionFinish es

```
#define BUFFER_SIZE 26
#define BUFFER_SIZE_TRANSMISION 32
#define COMMAND_SIZE 14
#define MIN_CMD_SIZE 6

/*Variables propias del main*/
int firstParameter;
int secondParameter;
char contadorDatosRx;
char *ptrReception;
bool receptionFinish;
char bufferReception[BUFFER_SIZE];
char bufferTrasmision[BUFFER_SIZE_TRANSMISION];
char command[COMMAND_SIZE];

const char helpMenu[] = {
    "\n\r 1) helpMenu - Print this help \n\r 2) setPin #pin newState - Set #pin to newState\n\r "
};
```

verdadera.

11. Dentro de la función main, antes de llamar a las funciones de configuración, inicialice la variable "receptionFinish" como false, la variable contadorDatosRx en cero y el puntero ptrReception apuntando al inicio del arreglo bufferReception.
12. Justo después del mensaje de bienvenida, agregue una línea que imprima el arreglo helpMenu.
13. Dentro del if() para la variable receptionChar, agregue una línea que retorne a la terminal el valor recibido por el micro a través de la recepción serial.

14. Justo después de agregar el envío del carácter, agregue un bloque switch-case para la variable receptionChar en el cual se analizaran 4 casos:

- Si receptionChar es igual a '\b'
- Si receptionChar es igual a '\r'
- Si receptionChar es igual a '\n'
- Si receptionChar es cualquier otro caso....

IMPORTANTE: Los casos \r y \n solo se dan juntos (porque son el <enter>), por lo cual se deja el caso de \r vacío (y sin el break), de forma que cuando llegue \r se ejecuta el caso \n.

15. A continuación se presentan 2 imagenes, las cuales corresponden a 2 de los 3 casos del switch-case que se deben programar. Agregue el código de la imagen en donde sea correcto (se deduce por el código que presenta la imagen y la funcionalidad que

```
{
    *ptrReception = '\0'; // Agregamos el caracter null para que tenga el formato de un String normal
    if (contadorDatosRx > MIN_CMD_SIZE) {
        parseCommand(&bufferReception[0], &command[0], &firstParameter, &secondParameter);
        receptionFinish = true;
    }

    // Como se recibio un comando, bueno o malo, debemos reiniciar el ptr para que reciba otro
    ptrReception = &bufferReception[0];

    // Devolvemos el contador a cero, para de nuevo contar los caracteres recibidos
    contadorDatosRx = 0;

    // Enviamos un LF+CR que es la representacion del <enter>
    sendString("\n\r");
}
break;
```

se desea obtener).

Y el otro código es

```

{
    if (ptrReception > &bufferReception[0]) {
        // Eliminamos del arreglo los dos elementos que se recibieron (el dato malo)
        *ptrReception-- = '\0';
        *ptrReception = '\0';
        // Borramos en la consola
        sendString(" \b");

        if (ptrReception == &bufferReception[0]) {
            ptrReception = &bufferReception[0];
            contadorDatosRx = 0;
        }
    }
}
break;

```

16. El último caso debe ser programado por usted. Tenga en cuenta:

- Los datos que se reciben deben ser contados allí (es lo primero).
- Agregar el dato que se recibe al buffer de recepción utilizando para esto un puntero y verificando que el tamaño del buffer siempre sea menor al número de datos recibidos (es el segundo paso).
- Finalmente, si sucede que el contador de elementos recibidos es mas grande que el tamaño del buffer, entonces debe disminuir en 1 el contador de datos, almacenar el dato en la posición a la que apunta el puntero sin aumentar el puntero para una siguiente posición y enviar a la consola 2 veces el símbolo del backspace y luego el símbolo '\$', para indicarle al usuario que llegó al límite del arreglo y que no es posible escribir mas.

17. Finalmente, después de los casos del switch-case, justo después de terminar este bloque, debe asignar el carácter NULL a la variable receptionChar, para que solo se vuelva a ejecutar esta tarea cuando se reciba un nuevo dato.

18. La siguiente tarea, que es la correspondiente a receptionFinish se muestra en la imagen.

```
if (receptionFinish == true) {
    if (!strcmp(command, "helpMenu")) {
        sendString(helpMenu);
    }
    if (!strcmp(command, "setPin")) {
        sendString(command);
        sendString("\n\r");
        setPinState(&firstParameter, &secondParameter);
    }
    receptionFinish = false;
}
```

19. Observe que allí se encuentra descrito el funcionamiento de dos comandos, uno llamado helpMenu y el otro llamado setPin. De esta misma forma deben ser los otros comandos. Observe además que la función setPinState recibe punteros a variables y no a las variables propiamente dichas, así deben ser todas las funciones que se le pide que sean creadas.

20. Las funciones (y sus comandos en consola) que debe crear son:

- Una función que le cambie el periodo en 'ms' el parpadeo del led de estado. Debe ser posible tener varios estados.
- Una función que permita controlar la frecuencia y el duty de una señal PWM. La frecuencia debe poder variar entre 3KHz y 10KHz (pueden ser valores enteros: 3, 4, 5...), además, a el duty se le debe poder asignar un valor entre 10% a 90%, también con pasos enteros de 10, o sea 10, 20, 30, etc.
- Una función que presente una conversión ADC sencilla en el PIN AN0, en el PIN AN1 y en el PIN AN2 (hacer una función para cada pin, por ejemplo getAN0 para el valor de AN0 y getAN1 para el valor de AN1). Hacer la

conversión en modo sleep. Utilice una tabla de lookup para los datos obtenidos, de forma que se entreguen los valores en mV.

- Una función que me entregue el valor de una captura de frecuencia, de una señal que puede variar entre los 5KHz a los 10KHz. **NO** utilice una tabla de lookup para la captura de frecuencia. Imprima el valor obtenido y la información necesaria para interpretar el dato, o sea, cada cuánto se está incrementando el Timer y una explicación de cómo se debe hacer la multiplicación (o sea, imprima un String con esta explicación).

Todo lo descrito en esta sección, son procedimientos que de alguna forma ya se realizaron, ya fuera en clase, en tareas o en talleres con los monitores. La única diferencia es la velocidad a la que se están realizando las tareas, debido al Cristal+PLL

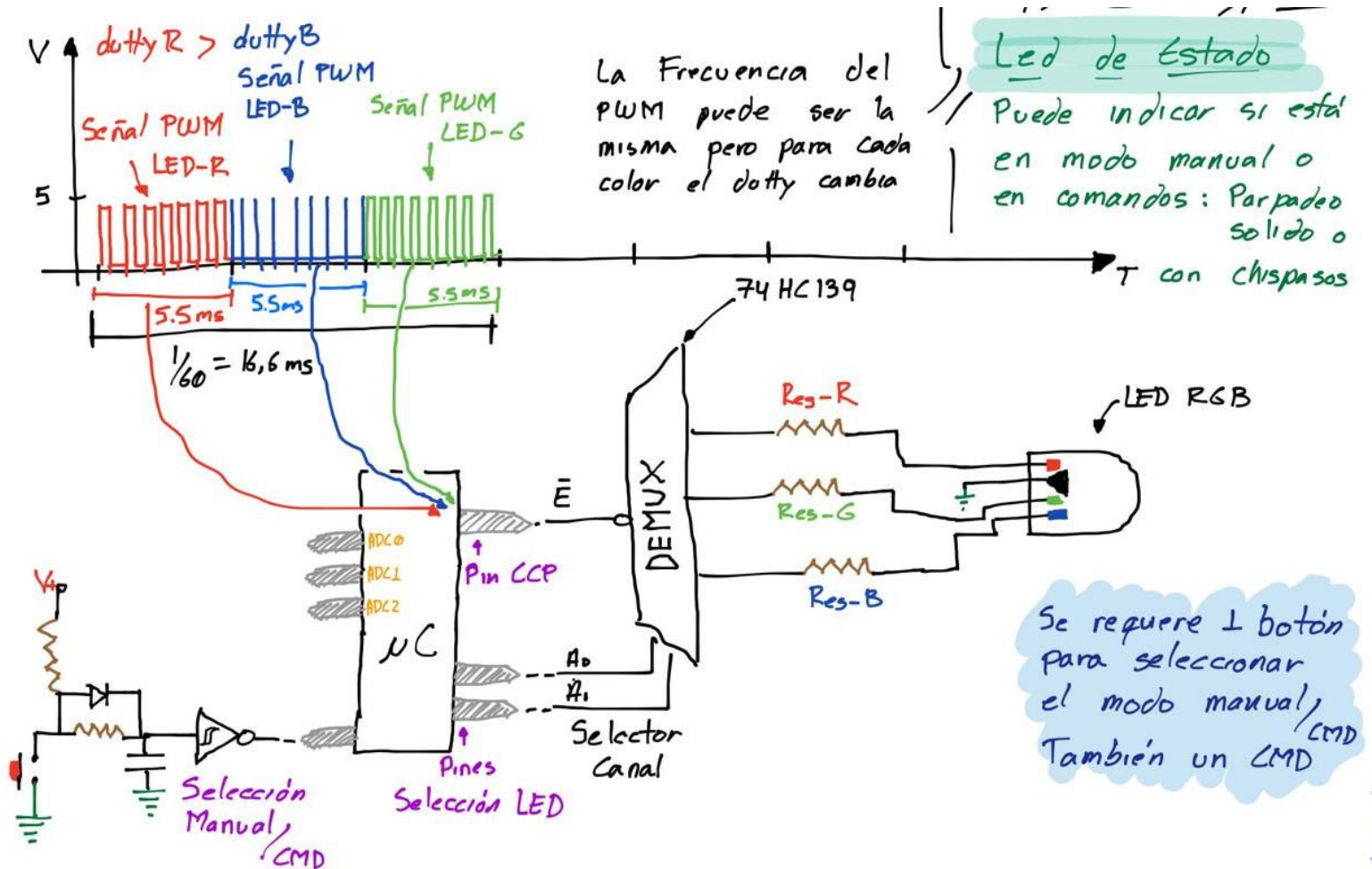
Parte B - 50%

La idea es construir un sistema que nos permita controlar el color generado por un Led RGB (son 3 leds en uno, cada uno de un color primario **Red**, **Green**, **Blue**). Para controlar la potencia que se le entrega a cada led, se va a utilizar el PWM que ya se configuró.

Ahora, recuerde que vista humana funciona a 24 cuadros/segundo (24FPS), por lo cual, para que el color generado por el LedRGB dé la sensación de ser un solo color y no tres, se debe hacer un refresco de estos tres leds a esa velocidad o superior (24FPS). Para nuestro caso, vamos a hacer un refresco a 60Hz o sea, cada color será actualizado 60 veces en 1 segundo. Lo cual implica que $1/60 = 16,6\text{ms}$ (tiempo para refrescar los tres leds) y por tanto cada led tiene un de refresco equivale a 5.5ms.

El ejercicio consiste en utilizar tres potenciómetros para generar diferentes colores con un LedRGB, donde cada potenciómetro controla la intensidad de un color específico (R, G o B). La intensidad de la lectura en el potenciómetro, debe ser transformada a un valor de Duty de la señal PWM, por lo cual es

necesario tener tres conversores ADC (uno por potenciómetro por color) y una única salida de PWM, la cual será demultiplexada



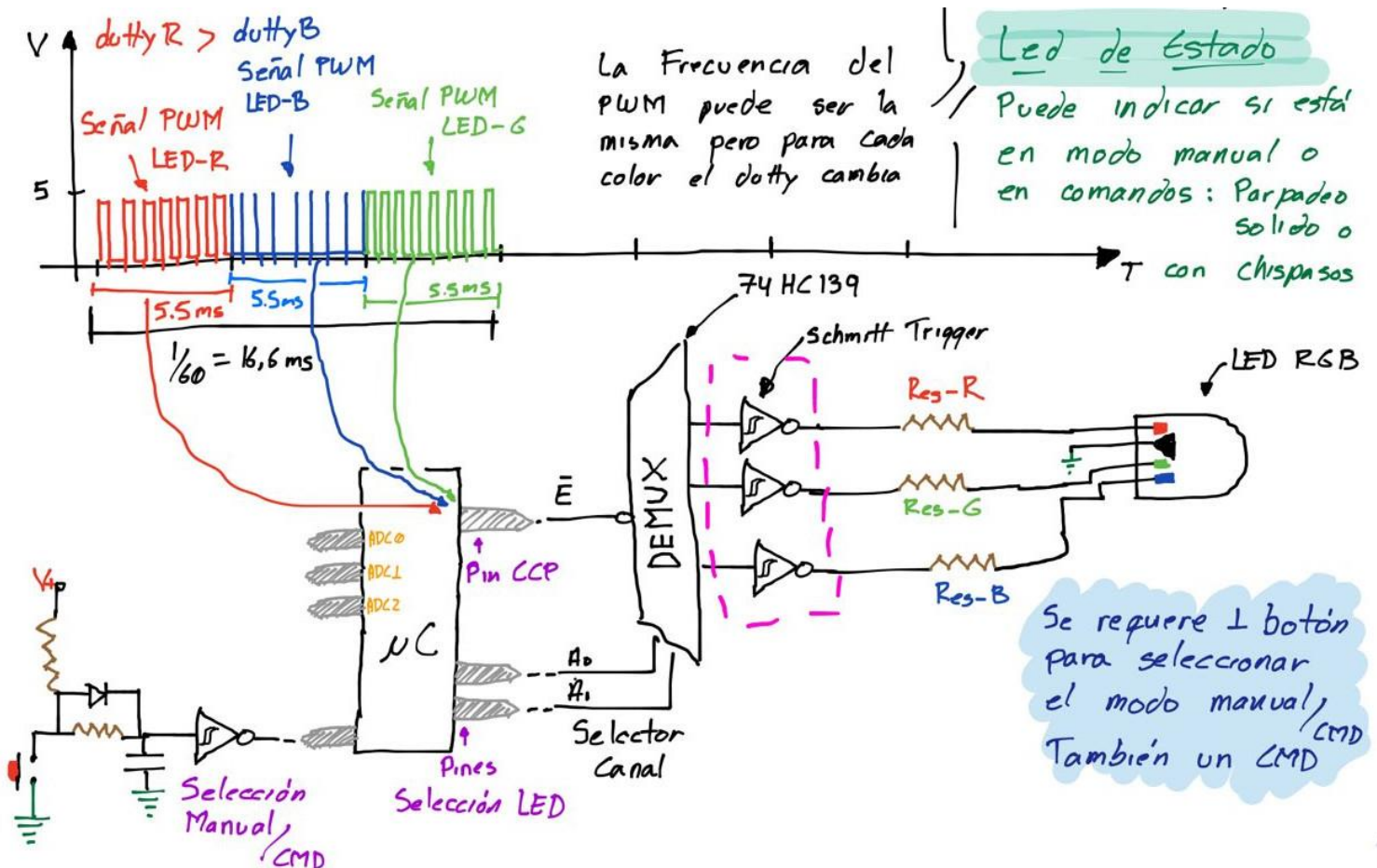
para llevar la misma señal a cada color.

Imagen #3 - Explicación inicial del ejercicio

Sucede que se deben tener en cuenta varias cosas:

1. El demultiplexor empleado mantiene los pines no utilizados en estado alto, lo cual hace que mientras ese pin se encuentra inactivo, se esté alimentando a el LedX (lo cual no es lo deseado), por lo cual, se tendrían dos led encendidos siempre a tope y solo un led alimentado con la señal PWM. Lo que se desea es que en cada momento solo tengamos un LedX alimentado por el PWM y los otros apagados. Para

solucionar esto, debemos utilizar un inversor, el cual mantiene dos de los 3 leds siempre apagados y solo uno encendido por el espacio de los 5.5ms



correspondientes.

Imagen #4 - Modificación, agregando inversor.

En la imagen 4, observamos la inclusión del inversor (Schmitt Trigger) a la salida del demultiplexor y antes de las resistencias para cada led: **Res_R**, **Res_G**, **Res_B**.

- La introducción del inversor (en nuestro caso el Schmitt Trigger) genera un efecto colateral y es que el periodo del Duty del PWM se invierte, o sea, si deseamos generar un PWM con un Duty de 25%, la señal real que el micro debe generar es del 75%, para que al invertirla quede del 25%, o sea **Duty_[real] = 100% - Duty_[mCCP]**

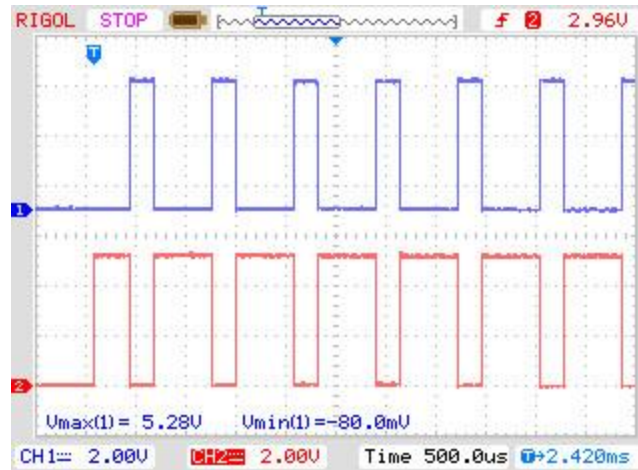


Imagen #5 - Señal PWM, en azul como sale del micro y en rojo como es invertida.

En la imagen 5 se observa este fenómeno de la inversión del PWM. Observe que al principio, la señal de salida hacia el led (la señal roja) es cero, sólo comienza mostrar la señal PWM cuando es seleccionada. En la imagen, la señal tiene una frecuencia de 1.22 KHz.

3. Debemos tener en cuenta como funciona la suma de colores. La teoría puede ser larga, así que para hacer la historia corta:
 - Cada color se representa como la suma de **RGB**.
 - Cada uno de ellos R, G y B tiene un valor de 0 (mínima intensidad) a 255 (máxima intensidad).
 - El color negro es ausencia completa de color $R=0, G=0, B=0$.
 - El color blanco es la suma de los tres colores a su máxima intensidad $R=255, G=255, B=255$.
 - La suma de las diferentes variaciones de los tres colores principales nos generan los otros colores, así por ejemplo, un color **naranja** es generado por la composición de $R=255, G=128, B=0$.
4. La intensidad de cada color es representada por el % del dutty, así un dutty de 0 es una intensidad 0 (no alumbra), mientras que un dutty de 100% implica un valor de 255 y/o un valor de 50% de dutty implica un valor de 128.

-
5. Como queremos crear colores, debemos ser capaces de cambiarle el Duty a la señal PWM al momento que vamos a alimentar el LedX específico (selección del canal en el demux), de esta forma, a cada refresco, estamos alimentando al LedX con el valor adecuado del Duty, para que genere su intensidad idónea.

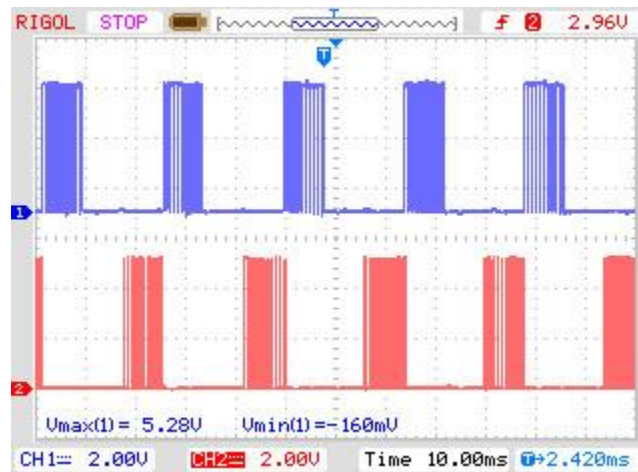


Imagen #6 - Señales PWM para el color Azul (en azul) y el color verde (en rojo).

Observe la imagen 6, se muestran las dos señales PWM que se inyectan a los leds Azul y Verde, el espacio que queda entre ambas señales es donde debe ir la señal del color rojo, la cual no se muestra porque se tiene un osciloscopio de solo dos puntas. En esta prueba, el tiempo de encendido para cada led era de 8ms.

6. Ahora, el tiempo de 5.5ms debe ser generado por un TimerX, diferente al utilizado para el Led de estado y al TMR2 (asignado al PWM). En su interrupción, se debe generar la orden para seleccionar el nuevo pin de salida, además de asignar el nuevo valor del Duty (específico de cada color).
7. Las conversiones ADC pueden ser lanzadas desde un TimerY, cada cierto tiempo, para mantener los datos actualizados. Al momento de ser utilizados, se toma el dato más "actualizado" y se lleva a una tabla de lookup, donde esta debe devolver el valor del Duty que se debe implementar en ese periodo de tiempo.

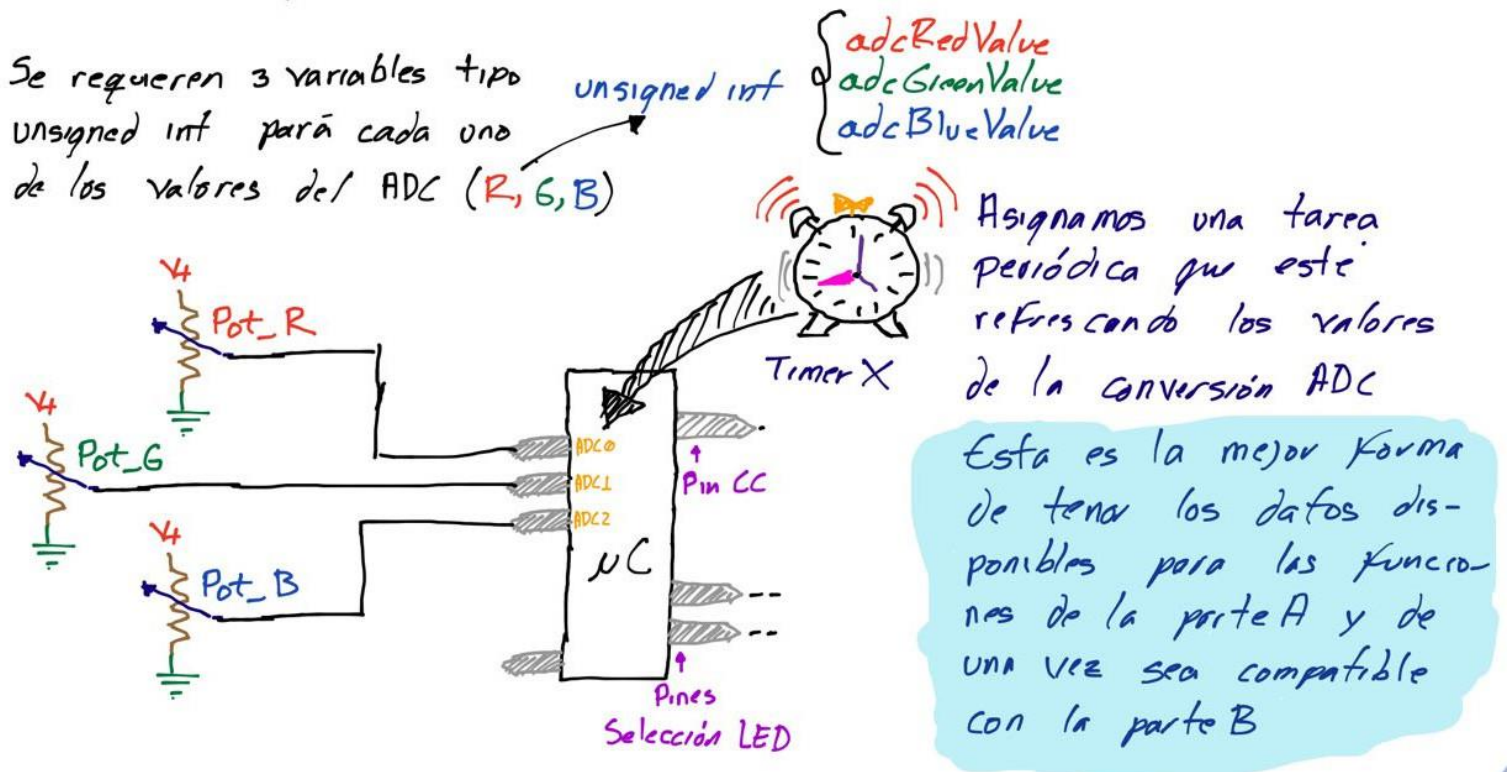
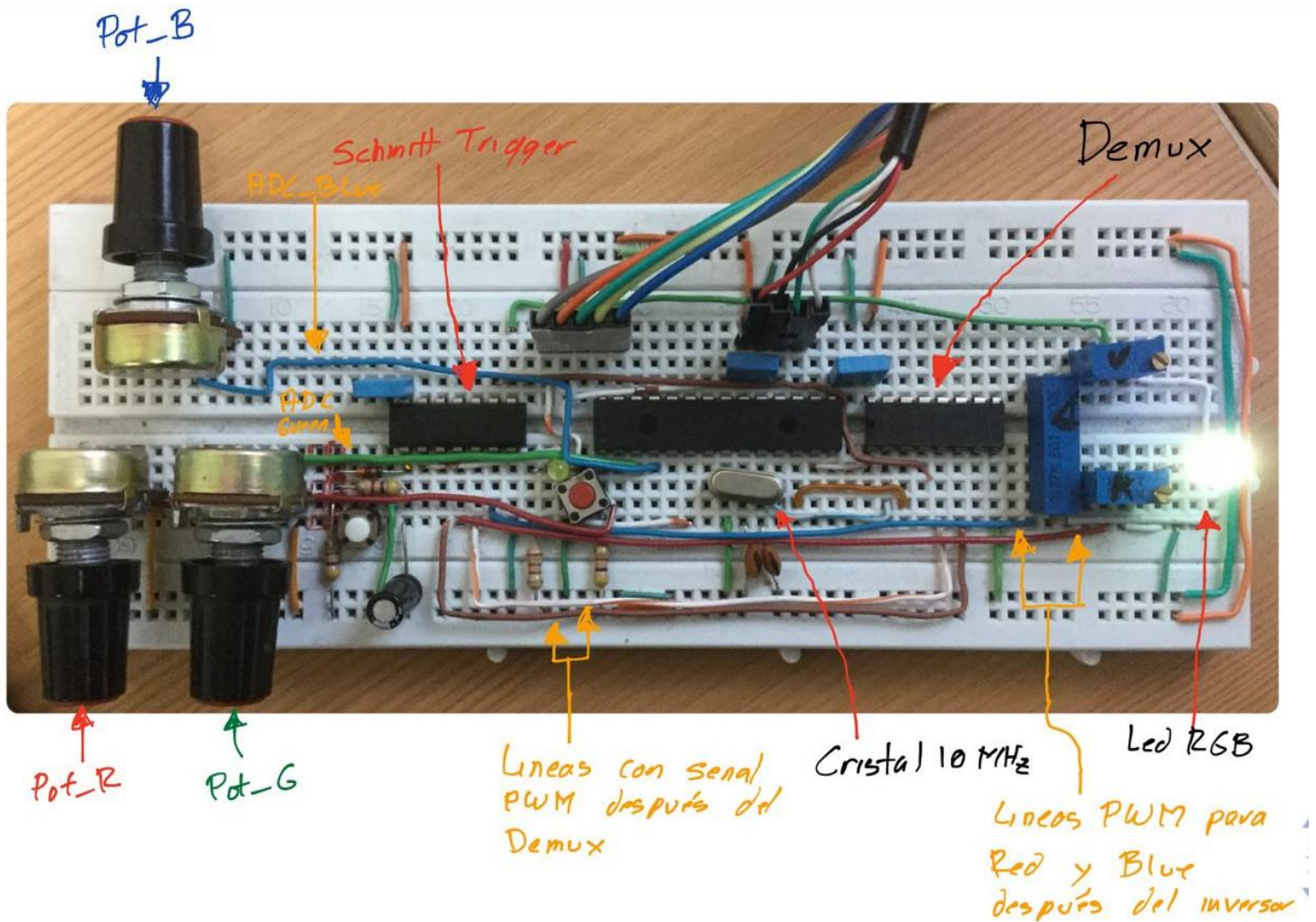


Imagen #7 - Conversión ADC como una tarea periódica disparada por un TimerX.

8. Observe que ambas tareas corren de forma independiente, por un lado se hacen las conversiones ADC y por el otro se está refrescando el LedRGB cada 5.5ms y la única parte donde se "unen" ambas aplicaciones es al momento de utilizar el dato de la conversión para un color y transformarlo a un dato de porcentaje de Dutty (que a su vez es la intensidad del color).
9. La resistencia que acompaña a cada led es diferente. Para que los colores sean correctos, se debe mantener una relación:
 - **LedRojo** -> 1.35mA
 - **LedVerde** -> 1.5mA
 - **LedAzul** -> 1.12mA

Estos valores de corrientes se deben encontrar antes de crear los colores, ya que la idea es garantizar que los tres leds brillen con la misma intensidad. Es válido tanto

poner un potenciómetro como cada resistencia, o encontrar el valor para cada



resistencia y ponerlo como un arreglo de varias resistencias en serie.

Imagen #8 - Montaje real funcionando.

10. Agregue una interrupción externa (puede ser RB0), para que con un botón se pueda seleccionar el modo de funcionamiento, si por consola (CMD) o manual (con los potenciómetros). Cuando el modo CMD esté habilitado, el proceso de la conversión ADC se debería detener y cuando el modo manual esté habilitado, el proceso de la conversión ADC debe funcionar normalmente.

-
11. Agregue un comando que permita seleccionar entre modo CMD y modo manual (o sea, lo mismo del punto 10 pero con comandos). Tenga en cuenta las mismas restricciones que para el punto 10.
 12. Agregue un comando que permita cambiar el valor de las variables de la lectura adc para cada uno de los potenciómetros (relacionado con el modo CMD), de esta forma, al cambiar estos valores, se cambia directamente el duty de cada una de las señales y por lo tanto se está modificando el color que muestra el Led desde la terminal.
 13. Presentar y disfrutar!

Observaciones:

- El led de estado debe estar presente.

Fecha de entrega:

Nerio Andrés Montoya: Hasta el Miércoles 3 de Mayo de 2017, antes del medio día.

Carlos A. Trujillo: Hasta el Miércoles 3 de Mayo de 2017, antes del medio día